FYP-I System Use Case Specification

for

SWE Autopilot

AI-Driven Autonomous Software Engineering Platform with Multi-Agent Development and Intelligent Requirement Change Management

Version 1.0 — Draft for Advisor Review

Prepared by

Abdul Wasay (Registration No. 2380101)
Ahmed Hassan Sheikh (Registration No. 2380102)

Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology University (SZABIST), Karachi Campus

Supervisor: Adil Ahmed Unar

Date created: October 21, 2026

Copyright © 1999 by Karl E. Wiegers. Template adapted for academic project documentation.

Table of Contents

Revision History **4**

Purpose of This Document **5**

4.1. User Registration and Authentication 6

4.1.1. Description: 6

4.1.2. Stimulus/Response Sequences: 6

4.1.3. Functional Requirements: 7

4.2. Requirement Analysis and Ambiguity Detection 8

4.2.1. Description: 8

4.2.2. Stimulus/Response Sequences: 8

4.2.3. Functional Requirements: 9

4.3. Requirement-to-Artifact Traceability 10

4.3.1. Description: 10

4.3.2. Stimulus/Response Sequences: 10

4.3.3. Functional Requirements: 11

4.4. Predictive Risk Analysis 12

4.4.1. Description: 12

4.4.2. Stimulus/Response Sequences: 12

4.4.3. Functional Requirements: 13

4.5. Architect/Critic Review and Approval 14

4.5.1. Description: 14

4.5.2. Stimulus/Response Sequences: 14

4.5.3. Functional Requirements: 15

4.6. Multi-Agent Code Generation 16

4.6.1. Description: 16

4.6.2. Stimulus/Response Sequences: 16

4.6.3. Functional Requirements: 17

4.7. Docker Sandbox Execution 18

4.7.1. Description: 18

4.7.2. Stimulus/Response Sequences: 18

4.7.3. Functional Requirements: 19

4.8. Initial Evidence-Based Verification 20

4.8.1. Description: 20

4.8.2. Stimulus/Response Sequences: 20

4.8.3. Functional Requirements: 21

4.9. Project Workspace and Artifact Management 22

4.9.1. Description: 22
4.9.2. Stimulus/Response Sequences: 22
4.9.3. Functional Requirements: 23
FYP-I Scope Boundary **24**
Advisor Review Checklist **24**
References **24**

Revision History

Name	Date	Reason For Changes	Version
Abdul Wasay / Ahmed Hassan Sheikh	October 21, 2026	Initial FYP-I baseline prepared for advisor review	1.0

Purpose of This Document

This document expands Section 4 of the SWE Autopilot SRS into system use cases using the exact organization of the supplied Sign Up Use Case Example. Every use case contains a description, actors, goal, preconditions, main flow, alternate flow, exception flow, postconditions, and functional requirements table. All statuses are Pending until advisor review.

4.1. User Registration and Authentication

4.1.1. Description:

The platform allows a project member to create an account and access the SWE Autopilot workspace using email/password authentication or configured Google and GitHub OAuth.

4.1.2. Stimulus/Response Sequences:

Use Case Name: User Registration and Authentication
Description: The platform allows a project member to create an account and access the SWE Autopilot workspace using email/password authentication or configured Google and GitHub OAuth.
Actors: Project Member; Authentication Service; Google OAuth; GitHub OAuth
Goal: Create or access a protected project workspace.
Pre-Conditions: The user is not authenticated; the backend and configured provider credentials are available for the selected method.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user opens the landing page and selects Sign In or Get Started. System Response: The system displays the authentication modal with email/password fields and Google/GitHub options. 2. User/Agent Action (Step 2): The user enters a name, email, and password for sign-up, or an email and password for sign-in. System Response: The system validates required fields, email format, and password length before submission. 3. User/Agent Action (Step 3): The user submits valid email/password data. System Response: The backend checks the account, hashes a new password for sign-up or verifies the stored hash for sign-in, creates an HTTP-only session, and returns the public user record. 4. User/Agent Action (Step 4): The user selects Google or GitHub. System Response: The system redirects the browser to the backend OAuth route, validates the callback state, links or creates the user, establishes a session, and returns the user to the frontend. 5. User/Agent Action (Step 5): The authenticated user selects Sign Out. System Response: The system clears the session and returns the user to the unauthenticated landing state.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: The user submits blank or invalid fields. System Response: The system highlights the invalid input and prevents submission until the data is corrected. Action/Condition: The email already exists during sign-up. System Response: The system displays an account-exists message and suggests signing in.
Exception Flow (System Errors)
Exception Condition: Database, provider, or network service is unavailable. System Response: The system displays a recoverable error, does not expose secrets, and keeps the user on the authentication screen. Exception Condition: OAuth state is missing, invalid, or expired. System Response: The backend rejects the callback and displays an authentication failure message.
Post Conditions: An authenticated user has a valid session and can create or open a project workspace; after sign-out, the session is cleared.

Table 1. User Registration and Authentication Stimulus/Response Sequences

4.1.3. Functional Requirements:

Req.ID	Description	Priority	Status
AUTH-01	The system shall display sign-up and sign-in forms with required name, email, and password fields as appropriate.	High	Pending
AUTH-02	The system shall validate email format, required fields, and a minimum password length of eight characters before submission.	High	Pending
AUTH-03	The system shall prevent duplicate local accounts for the same normalized email address.	High	Pending
AUTH-04	The system shall hash passwords using a one-way password hashing function and shall never store plaintext passwords.	High	Pending
AUTH-05	The system shall support configured Google and GitHub OAuth login through server-side callback routes.	High	Pending
AUTH-06	The system shall create an HTTP-only authenticated session containing only the application user identifier.	High	Pending
AUTH-07	The system shall display actionable errors for invalid credentials, provider failures, and unavailable services.	High	Pending

Table 1. User Registration and Authentication Functional Requirements

4.2. Requirement Analysis and Ambiguity Detection

4.2.1. Description:

The Requirement Analyzer converts a natural-language software requirement into structured requirement artifacts and identifies ambiguity, incompleteness, conflicts, and missing acceptance criteria.

4.2.2. Stimulus/Response Sequences:

Use Case Name: Requirement Analysis and Ambiguity Detection
Description: The Requirement Analyzer converts a natural-language software requirement into structured requirement artifacts and identifies ambiguity, incompleteness, conflicts, and missing acceptance criteria.
Actors: Project Member; Requirement Compiler Agent; Agent Runtime
Goal: Produce a reviewed, structured requirement baseline for a project.
Pre-Conditions: The user is authenticated, a project workspace exists, and the requirement text is available.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user opens a project workspace and enters a natural-language requirement. System Response: The system displays the requirement editor and accepts the text for analysis. 2. User/Agent Action (Step 2): The user selects Analyze Requirement. System Response: The Agent Runtime creates an analysis run, validates the input, and invokes the Requirement Compiler Agent. 3. User/Agent Action (Step 3): The compiler processes the requirement. System Response: The system extracts functional requirements, nonfunctional requirements, user stories, acceptance criteria, IDs, entities, and likely dependencies. 4. User/Agent Action (Step 4): The analysis completes. System Response: The system highlights ambiguous statements, missing information, conflicts, and incomplete acceptance criteria with explanations and severity. 5. User/Agent Action (Step 5): The user reviews or edits the findings. System Response: The system stores the accepted requirement baseline and records the review state and version.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: The requirement is empty or too short. System Response: The system requests meaningful requirement text and does not start an LLM analysis. Action/Condition: The user rejects a generated interpretation. System Response: The system preserves the original input, marks the interpretation as rejected, and allows revision and re-analysis.
Exception Flow (System Errors)
Exception Condition: The model provider is unavailable or returns an invalid response. System Response: The runtime marks the run as failed, records the diagnostic, and does not create unvalidated requirement artifacts. Exception Condition: The response violates the expected schema. System Response: The system rejects the response and requests a bounded retry or human review.
Post Conditions: The project contains a versioned requirement analysis with structured artifacts, identified ambiguities, and a visible review status.

Table 2. Requirement Analysis and Ambiguity Detection Stimulus/Response Sequences

4.2.3. Functional Requirements:

Req.ID	Description	Priority	Status
REQ-AN-01	The system shall accept a natural-language requirement for a selected project.	High	Pending
REQ-AN-02	The system shall create unique IDs for extracted requirements, user stories, and acceptance criteria.	High	Pending
REQ-AN-03	The system shall classify extracted items as functional or nonfunctional requirements where applicable.	High	Pending
REQ-AN-04	The system shall identify ambiguous, incomplete, conflicting, or underspecified requirement statements.	High	Pending
REQ-AN-05	The system shall explain each ambiguity or missing information finding in user-readable language.	High	Pending
REQ-AN-06	The system shall allow the user to review, edit, accept, or reject generated requirement artifacts.	High	Pending
REQ-AN-07	The system shall preserve the original requirement text and the version history of analysis results.	High	Pending

Table 2. Requirement Analysis and Ambiguity Detection Functional Requirements

4.3. Requirement-to-Artifact Traceability

4.3.1. Description:

The Traceability Manager links requirements to user stories, architecture components, APIs, code artifacts, tests, documentation, and verification evidence.

4.3.2. Stimulus/Response Sequences:

Use Case Name: Requirement-to-Artifact Traceability
Description: The Traceability Manager links requirements to user stories, architecture components, APIs, code artifacts, tests, documentation, and verification evidence.
Actors: Project Member; Traceability Service; Agent Runtime
Goal: Make the relationship between a requirement and downstream engineering evidence visible and reviewable.
Pre-Conditions: A project contains at least one analyzed requirement and one or more generated or manually linked artifacts.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user opens the Traceability view. System Response: The system loads the project graph and displays requirement IDs and linked artifacts. 2. User/Agent Action (Step 2): The system compiles or updates artifacts. System Response: The Traceability Service creates or updates links such as REQ → US → ARCH → API/COMPONENT → CODE → TEST → EVIDENCE. 3. User/Agent Action (Step 3): The user selects a requirement or artifact. System Response: The system shows direct links, dependent links, artifact versions, and missing downstream links. 4. User/Agent Action (Step 4): The user adds or corrects a relationship. System Response: The system validates the relationship, saves it with the current project version, and records the change.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: An artifact cannot be uniquely identified. System Response: The system requests a valid artifact ID or marks the relationship as unresolved. Action/Condition: A downstream artifact is missing. System Response: The system shows a traceability gap and identifies the next artifact expected by the workflow.
Exception Flow (System Errors)
Exception Condition: The graph store or database is unavailable. System Response: The system displays the last saved traceability snapshot and reports that new changes could not be saved. Exception Condition: A circular or invalid relationship is submitted. System Response: The system rejects the relationship and explains the allowed relationship types.
Post Conditions: The project has a queryable, versioned traceability matrix with visible gaps and evidence links.

Table 3. Requirement-to-Artifact Traceability Stimulus/Response Sequences

4.3.3. Functional Requirements:

Req.ID	Description	Priority	Status
TRACE-01	The system shall display requirement-to-artifact relationships for the selected project.	High	Pending
TRACE-02	The system shall support relationships between requirements, user stories, architecture, APIs/components, code, tests, documentation, and evidence.	High	Pending
TRACE-03	The system shall preserve stable IDs and version references for linked artifacts.	High	Pending
TRACE-04	The system shall identify missing or unresolved links in the traceability chain.	High	Pending
TRACE-05	The system shall allow an authorized user to add, edit, or remove a relationship.	High	Pending
TRACE-06	The system shall record relationship changes in the project audit history.	High	Pending

Table 3. Requirement-to-Artifact Traceability Functional Requirements

4.4. Predictive Risk Analysis

4.4.1. Description:

The Predictive Risk Analyzer evaluates requirements and proposed architecture for technical, architectural, performance, security, dependency, ambiguity, and implementation-complexity risks.

4.4.2. Stimulus/Response Sequences:

Use Case Name: Predictive Risk Analysis
Description: The Predictive Risk Analyzer evaluates requirements and proposed architecture for technical, architectural, performance, security, dependency, ambiguity, and implementation-complexity risks.
Actors: Project Member; Risk Analyzer Agent; Agent Runtime
Goal: Identify important risks before implementation begins and provide mitigation guidance.
Pre-Conditions: A reviewed requirement baseline and, where available, an architecture proposal exist.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user selects Run Risk Analysis. System Response: The runtime creates a risk-analysis run and collects the relevant requirements, artifacts, dependencies, and architecture records. 2. User/Agent Action (Step 2): The Risk Analyzer evaluates the project. System Response: The system identifies risk factors, affected artifacts, likelihood, impact, rationale, and suggested mitigation. 3. User/Agent Action (Step 3): The analysis completes. System Response: The system groups risks by category and sorts them by severity or priority. 4. User/Agent Action (Step 4): The user reviews a risk item. System Response: The system shows the evidence or artifacts that led to the risk and allows the user to acknowledge, annotate, or mark it for follow-up.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: The user runs analysis without a requirement baseline. System Response: The system requests requirement analysis first and does not produce an ungrounded risk report. Action/Condition: The user marks a risk as accepted. System Response: The system records the acceptance rationale and keeps the risk visible in project history.
Exception Flow (System Errors)
Exception Condition: The model provider or dependency graph is unavailable. System Response: The system records a partial or failed analysis and identifies the missing input instead of claiming that no risks exist. Exception Condition: A risk response is malformed. System Response: The runtime rejects the response, records the schema error, and requests review or a bounded retry.
Post Conditions: The project contains a categorized, prioritized risk register with rationale, affected artifacts, mitigation suggestions, and review status.

Table 4. Predictive Risk Analysis Stimulus/Response Sequences

4.4.3. Functional Requirements:

Req.ID	Description	Priority	Status
RISK-01	The system shall run a risk analysis for a selected project baseline.	High	Pending
RISK-02	The system shall evaluate at least architecture, technical, performance, security, dependency, ambiguity, and implementation-complexity risk categories.	High	Pending
RISK-03	The system shall assign a severity or priority to each identified risk.	High	Pending
RISK-04	The system shall identify the requirement or artifact affected by each risk where possible.	High	Pending
RISK-05	The system shall provide a rationale and mitigation suggestion for each risk.	High	Pending
RISK-06	The system shall allow an authorized user to acknowledge, annotate, or defer a risk with a recorded rationale.	High	Pending

Table 4. Predictive Risk Analysis Functional Requirements

4.5. Architect/Critic Review and Approval

4.5.1. Description:

The Architect Agent proposes a system architecture and the Critic Agent evaluates it for security, scalability, complexity, missing components, dependency problems, and requirement mismatches.

4.5.2. Stimulus/Response Sequences:

Use Case Name: Architect/Critic Review and Approval
Description: The Architect Agent proposes a system architecture and the Critic Agent evaluates it for security, scalability, complexity, missing components, dependency problems, and requirement mismatches.
Actors: Project Member; Architect Agent; Critic Agent; Agent Runtime; Advisor/Reviewer
Goal: Produce an approved architecture decision with alternatives, critique, rationale, and an auditable approval state.
Pre-Conditions: A structured requirement baseline and relevant risk analysis are available.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user requests architecture proposals. System Response: The runtime provides the approved requirements and risk findings to the Architect Agent. 2. User/Agent Action (Step 2): The Architect Agent creates one or more alternatives. System Response: The system displays components, APIs, data design, technology choices, communication patterns, assumptions, and trade-offs. 3. User/Agent Action (Step 3): The Critic Agent reviews the proposal. System Response: The system displays findings related to security, scalability, complexity, dependencies, missing components, and requirement coverage. 4. User/Agent Action (Step 4): The user or advisor reviews the debate. System Response: The runtime records discussion messages, revisions, and the selected architecture decision. 5. User/Agent Action (Step 5): An authorized reviewer approves or rejects the decision. System Response: The system records the approval result and allows code generation only for an approved architecture.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: The critic identifies blocking issues. System Response: The system returns the proposal to the architecture-review state and requires revision before approval. Action/Condition: The reviewer rejects the proposal. System Response: The system records the rejection reason and allows a new proposal or revision.
Exception Flow (System Errors)
Exception Condition: The architecture agents fail or return incomplete output. System Response: The runtime stores the failure evidence and prevents approval of an incomplete proposal. Exception Condition: The approval service is unavailable. System Response: The system preserves the pending decision and shows that approval has not been completed.
Post Conditions: The project contains an architecture decision record with proposal, critique, rationale, reviewer, timestamp, and approval status.

Table 5. Architect/Critic Review and Approval Stimulus/Response Sequences

4.5.3. Functional Requirements:

Req.ID	Description	Priority	Status
ARCH-01	The system shall generate at least one architecture proposal from the reviewed project requirements.	High	Pending
ARCH-02	The system shall represent components, APIs, data stores, technology choices, and communication patterns in a structured proposal.	High	Pending
ARCH-03	The Critic Agent shall evaluate security, scalability, complexity, missing components, dependencies, and requirement coverage.	High	Pending
ARCH-04	The system shall allow comparison of architecture alternatives and record trade-offs.	High	Pending
ARCH-05	The system shall store an architecture decision record with rationale and review history.	High	Pending
ARCH-06	The system shall require an approval state before an architecture proposal can be used for controlled code generation.	High	Pending

Table 5. Architect/Critic Review and Approval Functional Requirements

4.6. Multi-Agent Code Generation

4.6.1. Description:

The platform uses specialized agents to generate frontend, backend, API, database, configuration, test, and documentation artifacts while the Agent Runtime controls tools, permissions, repositories, and versions.

4.6.2. Stimulus/Response Sequences:

Use Case Name: Multi-Agent Code Generation
Description: The platform uses specialized agents to generate frontend, backend, API, database, configuration, test, and documentation artifacts while the Agent Runtime controls tools, permissions, repositories, and versions.
Actors: Project Member; Coding Agents; Agent Runtime; LLM Provider
Goal: Generate a coherent initial application implementation from approved requirements and architecture.
Pre-Conditions: An approved architecture decision exists, the target repository is accessible, and agent/model configuration is available.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user selects Generate Code. System Response: The system shows the approved requirements, architecture, target repository, agent plan, and requested artifact types. 2. User/Agent Action (Step 2): The user confirms the generation scope. System Response: The Agent Runtime creates a run, assigns specialized agents, and grants only the required tools and artifact permissions. 3. User/Agent Action (Step 3): Agents generate artifacts. System Response: The runtime captures prompts, outputs, file changes, dependency requests, and agent execution status. 4. User/Agent Action (Step 4): Generation completes. System Response: The system creates a versioned artifact set and shows a summary/diff for review. 5. User/Agent Action (Step 5): The user reviews the generated changes. System Response: The system allows approval for sandbox execution or rejection for revision.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: The user requests generation without an approved architecture. System Response: The system blocks generation and identifies the missing approval. Action/Condition: An agent requests an unauthorized tool or file. System Response: The runtime denies the request and records the permission decision.
Exception Flow (System Errors)
Exception Condition: An LLM provider is unavailable or returns an invalid artifact. System Response: The run is marked incomplete, the generated output is not treated as verified, and the user receives a diagnostic. Exception Condition: A repository write fails. System Response: The system preserves the proposed change set and reports that the repository was not updated.
Post Conditions: The project contains a versioned generated artifact set with agent history, permissions, diffs, and an approval state for execution.

Table 6. Multi-Agent Code Generation Stimulus/Response Sequences

4.6.3. Functional Requirements:

Req.ID	Description	Priority	Status
CODE-01	The system shall create a generation plan from approved requirements and architecture artifacts.	High	Pending
CODE-02	The system shall support specialized agents for frontend, backend/API, database/configuration, tests, and documentation artifacts.	High	Pending
CODE-03	The Agent Runtime shall enforce tool, repository, and artifact permissions for each agent.	High	Pending
CODE-04	The system shall preserve generated files, diffs, dependencies, prompts or references, and execution status as a versioned artifact set.	High	Pending
CODE-05	The system shall allow a user to review and approve or reject a generated change set before sandbox execution.	High	Pending
CODE-06	The system shall not label generated code as verified until it passes the verification workflow.	High	Pending

Table 6. Multi-Agent Code Generation Functional Requirements

4.7. Docker Sandbox Execution

4.7.1. Description:

The Docker Sandbox builds and runs generated applications in a controlled environment and collects process output, exit status, build results, and startup results.

4.7.2. Stimulus/Response Sequences:

Use Case Name: Docker Sandbox Execution
Description: The Docker Sandbox builds and runs generated applications in a controlled environment and collects process output, exit status, build results, and startup results.
Actors: Project Member; Agent Runtime; Docker Execution Service
Goal: Execute generated software safely enough to produce repeatable FYP-I build and runtime evidence.
Pre-Conditions: A generated artifact set is approved for execution and Docker is available.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user selects Run in Sandbox. System Response: The system validates the artifact version, execution policy, image configuration, and sandbox limits. 2. User/Agent Action (Step 2): The runtime starts the sandbox. System Response: The system builds or loads the configured Docker image with restricted permissions and mounted project artifacts. 3. User/Agent Action (Step 3): The container builds and starts the application. System Response: The system captures stdout, stderr, exit codes, health/startup output, duration, and resource-limit events. 4. User/Agent Action (Step 4): The execution completes or stops. System Response: The system stores the run evidence and returns a success, failure, timeout, or blocked status.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: Docker is not available. System Response: The system displays a setup error and does not attempt host execution. Action/Condition: A build fails. System Response: The system preserves logs and marks the run failed for diagnosis or future repair.
Exception Flow (System Errors)
Exception Condition: The container exceeds a configured limit. System Response: The runtime stops the container, records the limit event, and marks the run as stopped by policy. Exception Condition: The sandbox service crashes. System Response: The system records an infrastructure exception and prevents the result from being treated as successful.
Post Conditions: A controlled execution record exists with image/configuration details, logs, exit status, duration, and resource/policy events.

Table 7. Docker Sandbox Execution Stimulus/Response Sequences

4.7.3. Functional Requirements:

Req.ID	Description	Priority	Status
SBOX-01	The system shall execute approved generated artifacts inside a Docker sandbox.	High	Pending
SBOX-02	The sandbox shall apply configured restrictions for permissions, time, memory, storage, and process behavior where supported.	High	Pending
SBOX-03	The system shall capture build output, application startup output, standard output, standard error, exit code, and duration.	High	Pending
SBOX-04	The system shall prevent direct host execution when a sandbox execution is required.	High	Pending
SBOX-05	The system shall record timeout, resource-limit, policy-block, and infrastructure-failure events.	High	Pending
SBOX-06	The system shall associate each sandbox run with the exact generated artifact version and project.	High	Pending

Table 7. Docker Sandbox Execution Functional Requirements

4.8. Initial Evidence-Based Verification

4.8.1. Description:

The Initial Verification System evaluates generated software using available build, runtime, test, API, static-analysis, schema, and health evidence and reports whether the FYP-I verification conditions are satisfied.

4.8.2. Stimulus/Response Sequences:

Use Case Name: Initial Evidence-Based Verification
Description: The Initial Verification System evaluates generated software using available build, runtime, test, API, static-analysis, schema, and health evidence and reports whether the FYP-I verification conditions are satisfied.
Actors: Project Member; Verification Service; Pytest; Playwright; API Test Runner; Static Analyzer
Goal: Determine whether generated software is supported by executable verification evidence.
Pre-Conditions: A sandbox run exists and the project has configured verification checks or test commands.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user opens the Verification view. System Response: The system lists available checks for the selected project and generated artifact version. 2. User/Agent Action (Step 2): The user starts verification. System Response: The runtime schedules the selected checks and displays a running state. 3. User/Agent Action (Step 3): Checks execute. System Response: The system runs configured tests and analyses, captures commands, outputs, durations, and exit statuses. 4. User/Agent Action (Step 4): Verification completes. System Response: The system classifies each check as passed, failed, skipped, or unavailable and attaches the result to the relevant artifacts. 5. User/Agent Action (Step 5): The user reviews the result. System Response: The system shows evidence, unresolved failures, and whether the artifact can proceed to the next FYP-I review stage.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: No verification checks are configured. System Response: The system reports that verification is incomplete and does not claim a verified result. Action/Condition: A nonblocking check is skipped by policy. System Response: The system records the skip reason and displays it in the evidence summary.
Exception Flow (System Errors)
Exception Condition: A test runner or static analyzer is unavailable. System Response: The system records the unavailable dependency and marks the corresponding evidence as unavailable. Exception Condition: A check produces malformed output. System Response: The system stores the raw output, marks parsing as failed, and requires human review.
Post Conditions: The project has a verification summary with raw evidence, parsed results, linked artifacts, and an overall FYP-I verification state.

Table 8. Initial Evidence-Based Verification Stimulus/Response Sequences

4.8.3. Functional Requirements:

Req.ID	Description	Priority	Status
VER-01	The system shall list configured verification checks for a selected generated artifact version.	High	Pending
VER-02	The system shall execute available build, runtime, Pytest, Playwright, API, static-analysis, schema, and health checks as configured.	High	Pending
VER-03	The system shall capture the command, output, duration, exit status, and environment reference for each check.	High	Pending
VER-04	The system shall classify each check as passed, failed, skipped, or unavailable with a reason.	High	Pending
VER-05	The system shall associate verification evidence with the generated artifact and relevant traceability links.	High	Pending
VER-06	The system shall prevent an incomplete or failed verification run from being represented as a successful verification.	High	Pending

Table 8. Initial Evidence-Based Verification Functional Requirements

4.9. Project Workspace and Artifact Management

4.9.1. Description:

The platform provides a workspace for projects, artifacts, agent runs, approvals, versions, execution records, and verification evidence so FYP-I work remains organized and auditable.

4.9.2. Stimulus/Response Sequences:

Use Case Name: Project Workspace and Artifact Management
Description: The platform provides a workspace for projects, artifacts, agent runs, approvals, versions, execution records, and verification evidence so FYP-I work remains organized and auditable.
Actors: Project Member; Advisor/Reviewer; Agent Runtime
Goal: Provide one controlled place to review the current project state and its engineering history.
Pre-Conditions: The user is authenticated and has access to the workspace.
Basic Course of Events / Main Flow
1. User/Agent Action (Step 1): The user creates or opens a project. System Response: The system displays the project dashboard with status, scope, recent runs, risks, approvals, and unresolved issues. 2. User/Agent Action (Step 2): The user navigates to an artifact category. System Response: The system lists requirements, stories, architecture, code, tests, documentation, and evidence with IDs and versions. 3. User/Agent Action (Step 3): The user opens an artifact. System Response: The system shows content, relationships, history, owner, review state, and linked agent/execution records. 4. User/Agent Action (Step 4): The user or runtime creates a new version. System Response: The system records the version, timestamp, source action, and relationship changes.
Alternate Flow (Invalid Inputs / Review Rejection)
Action/Condition: The user lacks permission for a project. System Response: The system denies access and does not reveal project data. Action/Condition: An artifact is archived. System Response: The system hides it from active views while retaining its history and links.
Exception Flow (System Errors)
Exception Condition: The storage service is unavailable. System Response: The system shows a recoverable error and prevents misleading confirmation of unsaved changes. Exception Condition: Two updates conflict. System Response: The system identifies the version conflict and asks the user or runtime to reconcile it.
Post Conditions: The project has an organized, versioned workspace with auditable artifacts, runs, approvals, and evidence.

Table 9. Project Workspace and Artifact Management Stimulus/Response Sequences

4.9.3. Functional Requirements:

Req.ID	Description	Priority	Status
WORK-01	The system shall allow an authenticated project member to create and open a project workspace.	High	Pending
WORK-02	The system shall display project status, recent agent runs, risks, approvals, unresolved issues, and verification state.	High	Pending
WORK-03	The system shall store and retrieve versioned requirements, user stories, architecture, code, tests, documentation, and evidence artifacts.	High	Pending
WORK-04	The system shall associate artifacts with their creator, timestamp, version, and source workflow action.	High	Pending
WORK-05	The system shall enforce project-level access control for workspace data and actions.	High	Pending
WORK-06	The system shall preserve an audit history for approvals, permission decisions, execution runs, and artifact changes.	High	Pending

Table 9. Project Workspace and Artifact Management Functional Requirements

FYP-I Scope Boundary

The following proposal features remain FYP-II scope and are intentionally excluded from these FYP-I use cases: What-If Development Simulator, Change-Impact and Consistency Engine, Testing Agent and selective test replay, Closed-Loop Debugging and Recovery, Defensive Red-Team Agent, Attack Memory, Controlled Rollback, Release Gate, Deployment, final SDLC reporting, benchmarking, and final defense preparation.

Advisor Review Checklist

• Confirm every use case has a clear actor, goal, precondition, main flow, alternate flow, exception flow, and postcondition.

• Confirm requirements are testable, necessary, unambiguous, and correctly prioritized.

• Confirm authentication and workspace functions remain supporting platform functions rather than replacing the core FYP-I contribution.

• Update every Status column after advisor review and record the revision in the Revision History table.

References

1. SWE Autopilot FYP Proposal, October 7, 2026.

2. Software Requirements Specification Template (2025), supplied by the advisor.

3. Sign Up Use Case Example, supplied by the advisor.

4. SWE Autopilot repository: https://github.com/sheikh4467/SWE-Autopilot